SheetShrink: Efficient Spreadsheet Cleanup

Working draft for author review

***Abstract*—Spreadsheet packages can contain large populations of empty or redundant cell records that increase storage and ingestion costs. Cleanup must distinguish this structural redundancy from small formatting or dependency witnesses that preserve required workbook behavior. We present SheetShrink, a byte-oriented OOXML cleanup system that rewrites stored XLSX packages before opening them in Excel. It combines workbook-context analysis, local rewrite decisions, compact formatting representatives, and bounded parallel execution. We define its preservation contract and separate intentional policy choices from defects. This working manuscript reports bounded implementation checks and an ongoing native-rendering validation workflow. Controlled end-to-end performance results and final corpus coverage remain to be established.**

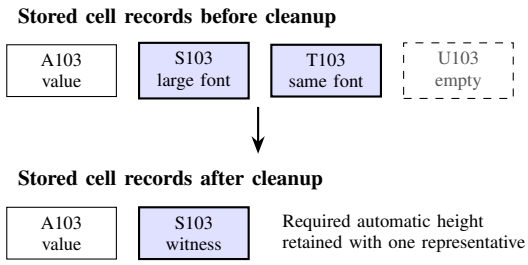


Fig. 1. Illustrative local cleanup. Numeric content and one required font witness remain. Redundant empty records are omitted while cell coordinates remain fixed. The preservation decision also depends on workbook context.

I. Introduction

Spreadsheet files are both data containers and executable visual documents. A user can store a compact table in a workbook whose worksheet parts contain large numbers of otherwise empty cells. Such records can increase the work required to decompress and interpret the package. Removing every empty-looking record is unsafe. An empty cell can preserve a border, override an inherited fill, or influence automatic row height through its font. A whitespace value can prevent text from extending into its neighbour. Cleanup therefore requires a model of the effects that stored structures have on the document.

For an XLSX input, SheetShrink makes these decisions from the stored package before loading the workbook in Excel. Its cleanup stage has no native rendering or calculated row-height observations. An Excel-hosted method can inspect the loaded layout and evaluate candidate edits against native observations. Our method instead retains compact source structures whose required effects follow from package analysis and the preservation rules. Native Excel is used later to evaluate the output. Legacy XLS conversion is a separately measured preparation stage.

Specialized spreadsheet parsing has demonstrated the importance of decompression, parsing, memory use and parallel execution for data loading [1]. Our problem adds a transformation obligation. The output remains an editable XLSX workbook and must retain required data, dependencies and visible presentation under a stated contract. File size alone is an inadequate measure of success. A smaller file that changes an important row height has failed that obligation.

Figure 1 illustrates the compact preservation approach. SheetShrink combines byte-oriented worksheet processing with compact preservation of necessary effects. It reuses existing styles and retains representative cells where those representatives preserve layout. Its production cleanup policy keeps whitespace blockers and applies dependency guards to eligible hidden constant tails. It leaves style and shared-string table compaction to a separate Excel save operation. This boundary keeps cleanup independent of the installed Excel application and makes its decisions inspectable in the package.

The performance objective includes the complete user operation. For a sufficiently bloated workbook, we seek to make cleanup followed by opening the cleaned file faster than directly opening its original. This objective must be measured, including the cases where cleanup adds overhead. A single-thread comparison separates improvements in work performed from gains due to extra processing resources. Parallel corpus experiments then examine throughput and memory.

The paper develops the preservation model, the small-footprint mechanisms that implement it, and an evaluation combining structural checks with native visual observations. The evaluation distinguishes completed observations, unsupported inputs and unresolved differences. Historical Python implementations provide context for the engineering progression. Their different cleanup rules require separate output and correctness accounting in any timing comparison.

II. Preservation and Comparison Contract

A. Observable spreadsheet behavior

A spreadsheet package contains more than the populated table that a user sees. Individual cell records can carry formatting, protection, references, or values that affect other cells. Conversely, the presence of a stored record does not establish that it contributes to the intended presentation. SheetShrink targets production cleanup of such packages while retaining the structures needed by its preservation contract. Aggressive historical configurations are separate test artifacts and are excluded from the production subject of this paper.

The relevant visual area includes cells with meaningful values or formulas, empty cells with effective visible fills or borders, and graphical objects. Formatting inherited from rows

or columns can contribute to this area. Conditional formatting and text extending into adjacent cells can also affect what is visible. The union of the original and transformed relevant areas is required when checking removed content. Layout changes in otherwise empty trailing rows or columns are permitted by the contract. Changes that move or alter relevant content remain subject to validation.

B. Information available before native loading

The cleanup decision uses stored values, formulas, styles, references and layout attributes. Native effective row heights and candidate before/after images are available only in the separate validation workflow. Stored custom heights can be retained directly. For automatically sized rows, a font-bearing blank can still require a representative even when its payload is empty.

An Excel-hosted alternative could measure a row, remove candidates and write the measured height as an explicit custom height. SpreadsheetML provides a *customHeight* attribute for that policy [2]. Applying this technique requires a trusted height for the chosen renderer and a contract for replacing automatic sizing with a fixed height. The evaluated cleanup derives its compact preservation decisions from package context.

C. Removal decisions and compact preservation

A record can be eligible for removal only after accounting for its required contribution. An empty cell may still influence automatic row height through its font. A whitespace-only value may stop text from extending from an adjacent cell. Referenced hidden content can influence displayed values and charts. These cases require different reasoning from a test for an absent payload.

The preservation strategy retains a small representative when that representative can preserve a required effect. Row-font sentinels and guarded promotion of an existing style into a row format are examples. Their applicability depends on inherited formatting, gaps, merges, protection, and the available style records. An unresolved case retains the required source representation. The objective is to remove eligible bloat throughout the package while keeping the preservation footprint small.

The production policy admits separately guarded hidden constant tails and retains ordinary nonempty hidden tables, including unreferenced values. Tail admission depends on dependency and layout checks [3]. Absence of a reference alone does not authorize general hidden-data removal. A future Excel-hosted cleanup stage could measure native layout before replacing additional sentinels or whitespace blockers. Such a stage is outside the implementation evaluated here.

D. Validation and attribution boundaries

Our fast visual validation compares independently rebuilt original and SheetShrink states using the same installed Excel environment. Ordinary-save and rebuild controls retain information for attributing differences introduced by Excel. For XLS sources, conversion precedes the XLSX cleanup subject and has its own failure accounting. A file requiring repair or inaccessible protected content receives an explicit unsuccessful outcome.

Targeted visual equality is an observation over the selected regions. We evaluate the selection mechanism separately with native reference captures and mutations. Unequal pixels are retained for investigation. Repeated rendering controls, volatile dependencies, and capture geometry help classify their cause. The results section must report the completed coverage and unresolved findings at its fixed evaluation cutoff.

Require: Workbook package W, cleanup policy P
Ensure: Cleaned package or explicit unsuccessful outcome
1: Check supported serialization of W
2: Resolve styles, strings, layout and references
3: **for all** worksheets in W **do**
4: Classify content and effective formatting
5: Identify protected references and layout anchors
6: Select guarded removals under P
7: Retain compact font and geometry witnesses
8: Rewrite admitted records and retain other effects
9: **end for**
10: Reconcile names and calculation-chain changes
11: Assemble output with unaffected package members
12: Publish only a successfully completed package

Fig. 2. SheetShrink's logical cleanup stages. Guards are resolved before removal. Worksheet scheduling and chunking preserve these decisions.

III. Method

A. Package-level analysis

Figure 2 summarizes the logical stages. Each line denotes a guarded operation whose preservation conditions are detailed below.

An XLSX file contains related ZIP members. SheetShrink first checks the supported serialization and reads the metadata needed to interpret worksheet decisions. The production parser targets unprefixed Transitional SpreadsheetML in its normal default namespace. Explicit prefixes on core cell structures, unsupported encodings and ambiguous representations can exclude an input. Exclusion retains a reason and does not establish that the workbook lacks redundant data.

The analysis resolves cell styles, shared strings, merged regions and dependency sources before applying the relevant removal rules. Worksheet cells are processed as byte ranges. This avoids constructing a document object model for every cell. It still requires correct recognition of attributes, hierarchy and relevant namespace boundaries. Unsupported syntax follows the documented fallback or eligibility path. Byte-oriented execution is an implementation strategy and does not itself prove semantic preservation.

B. Classify effects before removing records

A value, formula or inline-string payload is content. Numeric zero and shared formula followers require explicit treatment. For a candidate empty cell, the cleaner considers explicit formatting together with its row and column context. An apparently default style can cancel a visible inherited

format. Removing it would expose that format, so the effective effect matters.

A font can affect automatic row height without producing visible text. The cleaner therefore retains font representatives in its production mode. Where guards permit, an existing row style can replace repeated per-cell representations. This promotion must preserve other inherited components, gaps and merged cells. Dominance decisions include otherwise unsuitable styles, and ties have a fixed ordering. Ignoring an unsuitable majority would change the algorithm rather than merely accelerate it.

Whitespace cleanup is disabled in the production cleanup policy. A neighbour's text overflow and row sizing can depend on such values. Hidden constant-tail cleanup is separately guarded by references and layout conditions. Ordinary nonempty hidden tables remain outside that admitted tail policy. Protection and dependency decisions take precedence over a requested deletion.

C. Preservation structures at empty coordinates

The retained regression cases distinguish empty payloads from absent visual effects [3]. A default-format cell can mask a filled row or column. A font-only cell can determine automatic row height, including when a protection attribute accompanies the font. Merged text can require a local font representative even when no separate apply-font flag is present. These cases require resolving inheritance before classifying removable cells.

Preservation can also depend on several coordinates together. Center-across- selection formatting requires the contiguous blank span following its content cell. Keeping only the last blank loses the intended span. Row-level borders can require widths at empty column boundaries. A chart's linked number format can depend on the first, empty series cell. The guards therefore preserve specific layout and dependency anchors beyond the nonempty-cell envelope.

The hidden constant-tail rule admits only a dense horizontal sequence of at least 4,096 identical signed-integer cells, hidden and ending at column XFD, subject to reference and layout guards. Gaps, formulas, text and vertical runs remain outside this rule. The threshold identifies an admitted structural case; it does not establish that arbitrary repeated data are unintended.

D. Execution and output

The implementation distributes worksheet work and can subdivide large work. Scheduling and chunk size must preserve cleanup decisions. Workbook-level concurrency also determines how many decompression buffers, analysis contexts and pending outputs coexist. Consequently, throughput experiments include mixed workbook sizes and single-sheet workbooks, alongside large multi-sheet inputs.

The writer copies unaffected package members and can reuse compressed entries where supported. It does not generally renumber style or shared-string tables. The output writer must preserve input bytes and existing outputs on a rejected operation. A partially processed package is not published as a successful cleanup. These invariants are checked separately from compressed file size.

E. Native validation

The companion DeltaSieve manuscript describes the selection and capture methodology [4]. Its working PDF is included in the supplementary material at related-manuscripts/deltasieve.pdf. We use native Excel rendering to investigate selected potentially affected regions. Original and cleaned copies are rebuilt in the same renderer environment. A separate validation methodology identifies value, effective-style, layout and dependency candidates. Its selection scope and failure accounting are part of the reported evidence. Independent reference samples and injected changes are required to evaluate that selector. Equality of its selected images alone cannot establish equality of every possible spreadsheet observation.

IV. Evaluation

The evaluation asks three questions. RQ1 examines compressed package reduction and the remaining preservation footprint. RQ2 measures cleanup time, memory and the combined cleanup-plus-opening operation. RQ3 examines structural preservation and observed native presentation under the stated validation scope.

A. Corpus identity and outcome accounting

The evaluation treats Data.gov, Destatis and Enron as distinct populations. Archive checksums, member names and duplicate occurrences identify each selected corpus version. Legacy XLS conversion is reported separately from XLSX cleanup. A historical Enron development selection contains 20,872 inputs, with 20,503 prepared artifacts and 369 unsuccessful preparation outcomes at its recorded preparation checkpoint. Its selected visual scope contains 16,799 workbooks. This run ended after the worker-pool investigation. These counts describe that development selection and do not establish completed visual coverage. The replacement EDRM Enron v2 XML handover contains 34,065 distinct content-addressed candidates totaling 8.470 decimal GB of uncompressed payload. The imported selection unit is a distinct candidate SHA-256. Candidate transport was checked against the supplied manifests, and the import retained the original candidate bytes. This externally generated extraction includes legacy Excel and other OLE candidates. Native format recognition, successful XLSX conversion and visual eligibility remain separate evaluation stages. Candidate count therefore describes the imported input frame, with workbook outcomes to be reported after processing. The upstream extraction provenance and local import identities are retained alongside the experiment.

The two frozen XLSX inventories in Table I come from the versioned source profiles. Each listed archive member has an individual SHA-256 identity. Their source-frame sizes are separate from the eligible, completed and independently referenced subsets.

A dated evaluation cutoff will report compared workbooks, differences, failures, unsupported representations and unfinished work. Repeated attempts have their own counts. An

TABLE I
FROZEN XLSX SOURCE INVENTORIES. SIZES ARE DECIMAL GB OF LISTED WORKBOOK FILES AND DO NOT MEASURE COMPLETED NATIVE COVERAGE.

Corpus	XLSX members	XLSX total (GB)
Data.gov	9,117	25.273
Destatis	1,182	2.364

unchanged package can bypass visual work under an exact input-identity check, but a failed capture cannot contribute an equality observation. The full recursive Enron XML-derived corpus is a separately identified population.

B. Sampling and reference controls

The random evaluation sample is selected from a frozen eligible frame with a recorded seed before inspecting visual findings. Strata use corpus identity, package size and structural features available before validation. Low-reduction and unchanged files remain represented. A separately labelled development stress set contains large and historically problematic workbooks.

Feasible sampled files receive full native reference captures over the relevant visual area using a capture plan independent of the candidate selector. A full-reference timeout remains in the denominator. It is not replaced silently by an easier workbook. Mutations with known visible effects cover cell values, styles, geometry, conditional dependencies and objects. Mutations that Excel normalizes away are reported separately.

C. Performance and comparison subjects

The comparison records each method's input representation, Excel dependency, use of native layout observations, removal policy and resulting workbook properties. The first-opening objective measures cleanup followed by loading the resulting workbook against direct loading of the source. An Excel-hosted cleaner includes its required source opening in the corresponding workflow. Cleanup, opening, saving and any required reopening have separate phase timings under a common readiness endpoint. Repeated-use experiments report preparation cost and subsequent opening cost separately.

Production cleanup time covers package analysis and output construction. Native preservation-validation cost has its own accounting. A corpus workbook supplies an observed source state. Establishing preservation requires structural checks and independently prepared native controls. Synthetic stress cases vary eligible redundant records, inheritance masks, font witnesses, merged spans and protected references, with fixed-height and automatic-height rows distinguished. Report removed records and bytes together with retained witnesses, exclusions and preservation outcomes under each cleanup configuration.

For each input, report the original and cleaned compressed sizes and their ratio. Report distributions and corpus totals, including inputs with no reduction. Opening time uses a documented readiness signal and fresh owned Excel processes. Single-thread cleanup, cleanup with parallelism and corpus throughput are separate experiments. RAM measurements specify the sampling interval and whether they refer to startup, steady state or peak use.

Microsoft Check Performance is an optional cleanup comparison subject. Its Excel save-only control is necessary to distinguish optimization from serialization and recalculation effects. Its availability does not guarantee that all selected files can be processed. Historical DOM and regular-expression Python implementations have different cleanup decisions. A timing comparison must therefore include output differences and preservation outcomes for each version.

D. Present verified evidence and remaining measurements

Focused implementation verification includes 292 passing Core library tests and one ignored benchmark at the recorded development revision. A separate exploratory comparison of an attribute-scan proposal found identical decompressed outputs on seven hash-matched review inputs, over two alternating-order repetitions. The two large cases showed essentially unchanged wall time. The proposal was not adopted. These observations establish a bounded engineering result and do not estimate corpus-wide speedup or visual correctness.

Evaluation freeze pending. Populate the corpus reduction, end-to-end opening, resource and independent-reference result tables from frozen manifests. Development-run counters are not used to claim an absence of regressions.

E. Check Performance preparation checkpoint

A completed Destatis preparation campaign attempted all 1,182 source inputs and retained 1,016 prepared outputs and 166 failures. Its raw failure reasons contain 165 opening timeouts and one automation failure when setting Excel's Interactive property. These are preparation outcomes. Visual validation of the outputs and matched baseline cost measurements remain open.

Recovery checks of the same 166 frozen failure inputs used an owned opening-dialog monitor. The first three inputs completed in 13.213 seconds and the remaining 163 in 442.761 seconds, each batch with two workers. The checks identified Name Conflict dialogs for all 165 former opening timeouts. The Interactive-property failure recurred. A separate known-positive input produced its prepared output in 8.114 seconds. Source-identity reconciliation accounts for every original failure, while conflicting-name contents remain unresolved. Worker count changed during the first campaign, so its aggregate elapsed time is observational. Original and recovery receipts remain separately versioned alongside the 1,016 successful preparation artifacts.

V. RELATED WORK

A. Workbook bloat and cleanup tools

Microsoft Check Performance removes cells associated with unnecessary formatting and other stored information from a loaded workbook [5]. Its documentation warns that optimization can remove pixel-art cells. Inquire's Clean Excess Cell Formatting command removes excess cells beyond the last

nonblank cell and can change conditional-formatting coverage [6]. These documented policies motivate comparing both reduction and preservation outcomes under explicit contracts.

Package-level cleanup also predates SheetShrink. XLStylesTool's developer describes an Open-XML-based cleaner for excessive and corrupted styles that operates independently of Excel's object model [7]. NXPowerLite Desktop 10 lists excess XLSX cell-format cleanup among its features [8]. Its engine description discusses formatted blank cells as a source of large worksheet parts [9]. These are relevant comparison subjects. Their available descriptions do not establish the guarded removal rules, compact row-font witnesses or native preservation outcomes examined here. Those properties require version-specific evidence. SheetShrink's positioning therefore includes its effect-aware rules, retained representatives and measured costs alongside its execution before native loading.

B. Spreadsheet loading and specialized parsing

SheetReader studies specialized spreadsheet ingestion and the coupling of parsing, decompression and parallel execution [1]. It motivates evaluating loading cost and memory. SheetShrink produces an editable workbook whose required layout and dependencies must survive package rewriting. Its evaluation measures cleanup together with subsequent use of the output.

Mison combines structural indexing with speculative access to queried JSON fields [10]. Simdjson uses SIMD operations in a validating JSON parser [11]. These systems motivate efficient processing of semi-structured representations. SheetShrink's document-rewriting obligation additionally accounts for inheritance, references and native layout effects.

C. Data transformation and spreadsheet refactoring

Harris and Gulwani synthesize table transformations from example input/output pairs [12]. Potter's Wheel integrates interactive transformation and discrepancy detection [13]. HoloClean repairs data through probabilistic inference combining constraints and statistical evidence [14]. These systems infer or specify intended changes to data. SheetShrink's target is removal of admitted package redundancy under a preservation contract. Their data-repair accuracy measures therefore require a different ground truth from our reduction, cost and preservation measurements.

Hermans et al. detect formula smells and suggest refactorings to improve spreadsheet maintainability [15]. This is relevant to reasoning about workbook structure and behavioral obligations. Formula readability and refactoring objectives do not establish that empty cell records are removable while retaining native layout. Execution outside Excel alone likewise leaves the required effect analysis to be specified.

D. Validation and dependency analysis

Spreadsheet slicing integrates dependency reasoning with testing and debugging [16]. Dependencies also constrain removal when retained formulas or graphical objects refer to candidate content. Workbook comparison tools provide observations about cell and format differences [17]. Our native validation protocol uses DeltaSieve's region selection and capture workflow [4], with independent references and mutation controls to evaluate selected coverage. These validation mechanisms provide evidence about cleanup outcomes after the package transformation.

VI. Discussion and Limitations

The preservation contract is narrower than all possible spreadsheet behavior. Rendering depends on the installed Excel version, fonts and calculation state. A selected-region observation has limited scope. Unsupported XML and unsuccessful opening or saving remain visible outcomes. The independent reference sample estimates behavior on its eligible population and cannot establish a universal absence of regressions.

Large reductions can coexist with small but important preservation structures. A future Excel-hosted stage could replace font sentinels with measured custom row heights and validate the replacement immediately. Such a stage has a native application dependency and is outside the cleanup implementation examined here. Whitespace removal likewise requires a separate justification of overflow and layout effects. It is not assumed safe merely because a displayed value is blank.

The historical implementations explain the progression from diagnosis through regular-expression cleanup to specialized byte-oriented execution. They are useful engineering artifacts with distinct output behavior. Their age or slower runtime does not establish that the current algorithm is correct.

VII. Conclusion

SheetShrink addresses spreadsheet redundancy through effect-aware removal and compact preservation. Its implementation and evaluation distinguish compressed size, XML changes, execution cost and observed presentation. The completed manuscript must quantify those outcomes using the frozen experiments described above and state the remaining unsupported cases.

References

[1] H. Gavriilidis, F. Henze, E. T. Zacharatou, and V. Markl, "SheetReader: Efficient specialized spreadsheet parsing," *Information Systems*, 2023.

[2] Microsoft, "Row.CustomHeight property," Open XML SDK documentation, accessed November 9, 2026. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/documentformat.openxml.spreadsheet.row.customheight

[3] SheetShrink project, "XLSX shrinking behavior," Versioned software specification, revision a7eef1a, 2026. [Online]. Available: https://github.com/Alexander-Aue-Johr/excel-shrink-rust/blob/a7eef1a/docs/behavior/shrinker.md

[4] "DeltaSieve: Targeted visual validation of spreadsheet transformations," Companion working manuscript prepared for ICDE 2027, 2026, supplementary material, related-manuscripts/deltasieve.pdf.

[5] Microsoft, "Cleanup cells in your workbook," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/cleanup-cells-in-your-workbook

[6] ——, "Clean excess cell formatting on a worksheet in Excel for Windows," accessed November 9, 2026. [Online]. Available: https://support.microsoft.com/en-us/excel/clean-excess-cell-formatting-on-a-worksheet-in-excel-for-windows

[7] XLGeek, ".NET4.0 version of the XLStylesTool is now available," 2011, developer description, accessed November 9, 2026. [Online]. Available: https://sergeig888.wordpress.com/2011/03/21/net4-0-version-of-the-xlstylestool-is-now-available/

[8] M. Power, "What's new in NXPowerLite Desktop 10," vendor release description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/whats-new-in-nxpowerlite-desktop-10

[9] ——, "Compression engine major upgrade—version 10," vendor description, accessed November 9, 2026. [Online]. Available: https://neuxpower.com/blog/wecompress-file-compressor-engine-upgrade

[10] Y. Li, N. R. Katsipoulakis, B. Chandramouli, J. Goldstein, and D. Kossmann, "Mison: A fast JSON parser for data analytics," *Proceedings of the VLDB Endowment*, vol. 10, no. 10, 2017. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2017/05/mison-vldb17.pdf

[11] G. Langdale and D. Lemire, "Parsing gigabytes of JSON per second," *The VLDB Journal*, vol. 28, no. 6, pp. 941–960, 2019. [Online]. Available: https://arxiv.org/abs/1902.08318

[12] W. R. Harris and S. Gulwani, "Spreadsheet table transformations from examples," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2011, pp. 317–328. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2012/08/pldi11-table-synthesis-tr.pdf

[13] V. Raman and J. M. Hellerstein, "Potter's wheel: An interactive data cleaning system," in *Proceedings of the International Conference on Very Large Data Bases*, 2001, pp. 381–390. [Online]. Available: https://cs.brown.edu/courses/cs227/archives/2016/papers/PottersWheel.pdf

[14] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1190–1201, 2017. [Online]. Available: https://www.vldb.org/pvldb/vol10/p1190-rekatsinas.pdf

[15] F. Hermans, M. Pinzger, and A. van Deursen, "Detecting and refactoring code smells in spreadsheet formulas," *Empirical Software Engineering*, vol. 20, no. 2, pp. 549–575, 2015.

[16] J. Reichwein, G. Rothermel, and M. Burnett, "Slicing spreadsheets: An integrated methodology for spreadsheet testing and debugging," in *Proceedings of the Second Conference on Domain-Specific Languages*. USENIX, 1999, pp. 25–38. [Online]. Available: https://www.usenix.org/legacy/publications/library/proceedings/dsl99/full_papers/reichwein/reichwein_html/index.html

[17] Microsoft, "Basic tasks in spreadsheet compare," Microsoft Support, accessed November 8, 2026. [Online]. Available: https://support.microsoft.com/en-gb/excel/basic-tasks-in-spreadsheet-compare